

Prepared by: Amoy Turner
Project: CarFinder
Date: September 2026
Live Site: <https://carfinder-amoy.netlify.app/>

Purpose & Project Overview

This maintenance plan provides the procedures needed to keep CarFinder functional, secure, and up to date after project handoff. CarFinder is a React based web application that allows users to search vehicle inventory by year, make, and model, view vehicle details, save vehicles using Favorites, and add vehicles to the inventory. The application also uses the NHTSA vPIC API to provide vehicle make and model information.

Regular maintenance will include testing core functionality, monitoring the deployed application, reviewing analytics and SEO performance, checking dependencies and external services, and addressing reported issues. This plan is intended to provide future developers or maintainers with a clear process for keeping CarFinder operational.

Maintenance Responsibilities

The developer or individual responsible for maintaining CarFinder should regularly verify that the application remains available and that its main features continue to work correctly.

Maintenance should include reviewing the deployed website, source code, dependencies, analytics, SEO, external API functionality, and reported bugs.

Any changes to the application should be tested before being merged into the production branch. GitHub should continue to be used for version control, issue tracking, branches, and pull requests so that changes can be documented and reviewed before deployment.

Maintenance Schedule

Frequency	Maintenance Tasks
Weekly	Verify the live website is available, test navigation, search functionality, vehicle details, Add Vehicle, and Favorites, and review any reported bugs or issues.
Monthly	Review Google Analytics data, check Netlify deployment status, verify the NHTSA vPIC API is functioning, review application dependencies, and test the application on current browsers and different screen sizes.
Quarterly	Review SEO metadata and search performance, perform a complete functionality test, review and update documentation, check for outdated dependencies, and review the overall user experience.
As Needed	Fix reported bugs, update vehicle information or content, address API changes, update dependencies with security issues, make design improvements, and deploy approved feature updates.

Application & Functionality Maintenance

CarFinder's primary features should be tested regularly to ensure that updates or external changes have not affected the application. Testing should include searching inventory by year, make, and model; opening vehicle details; adding vehicles; saving and removing Favorites; clearing search filters; navigating between pages; and verifying that locally stored information persists as expected.

The application should also be checked on different screen sizes to make sure the responsive interface continues to display correctly. Any broken buttons, navigation issues, incorrect information, or unexpected errors should be documented and corrected.

NHTSA vPIC API Maintenance

CarFinder uses the NHTSA vPIC API to retrieve vehicle make and model information. Because this is an external service, its availability and functionality are outside of CarFinder's direct control.

The API should be tested monthly and whenever users report problems with vehicle search data. If NHTSA changes its API endpoints or response structure, the related CarFinder service code may need to be updated. Any API-related errors should be tested separately from CarFinder's locally managed inventory data before changes are made.

Deployment & Hosting Maintenance

CarFinder is deployed using Netlify and connected to the project's GitHub repository. The live application should be checked weekly to verify that it remains publicly accessible.

After code changes are approved and merged, the Netlify deployment should be reviewed to make sure the build completes successfully. If a deployment fails, the deployment logs should be reviewed before additional changes are made. The live application should also be tested after every production deployment.

Analytics & SEO Maintenance

Google Analytics is installed on CarFinder to monitor website traffic and user activity. Analytics should be reviewed monthly to identify changes in traffic, user engagement, and how visitors interact with the application.

CarFinder's SEO metadata should be reviewed quarterly or whenever major content or functionality changes are made. Page titles, descriptions, headings, and other metadata should continue to accurately describe each page. Search performance can also be used to determine whether keywords or content should be updated.

Dependencies & Security

CarFinder's project dependencies are managed through npm and are documented in the project's package.json and package-lock.json files. Dependencies should be reviewed monthly for outdated packages, security vulnerabilities, or compatibility issues.

Updates should not be applied directly to the production version without testing. Dependency changes should be completed in a separate Git branch, tested locally, and submitted through a pull request before being merged.

Sensitive information, credentials, or private keys should not be committed to the public GitHub repository. If future versions of CarFinder introduce services that require credentials, those values should be stored securely using environment variables.

Data & Browser Storage

CarFinder currently uses browser localStorage for locally managed application data, including user created inventory information and Favorites. Because this information is stored in the user's browser, it is not a centralized database and may be lost if the user clears browser data or uses another device or browser.

Future development may replace localStorage with a database or backend service if persistent multi-user data storage is required. If that change is made, the maintenance plan should be updated to include database backups, access control, recovery procedures, and database maintenance.

Backup & Recovery

GitHub serves as the primary source code repository and version history for CarFinder. Important code changes should be committed and pushed regularly so that previous versions can be recovered if necessary.

If a deployment introduces a serious problem, the previous working version can be identified through Git history and restored or redeployed. Project documentation and other handoff materials should also remain stored in the repository so they are available with the project source files.

Because the current application does not use a centralized production database, there is no separate database backup required at this time.

Issue Tracking & Updates

Bugs, maintenance requests, and future improvements should be documented using GitHub Issues. Each issue should clearly explain the problem or requested change and include enough information for another developer to reproduce or understand it.

Development work should be completed on a separate branch whenever possible. After testing is complete, changes should be submitted through a pull request and reviewed before being merged into the production branch. This process provides a record of changes and reduces the risk of introducing untested code into the live application.

Maintenance Resources

The following resources are used to operate and maintain CarFinder:

- **Source Code & Version Control:** GitHub
- **Hosting & Deployment:** Netlify
- **Vehicle Data:** NHTSA vPIC API
- **Analytics:** Google Analytics 4
- **Application Framework:** React
- **Build Tool:** Vite
- **Package Management:** npm
- **Local Data Storage:** Browser localStorage